# 36

# I/O 设备

在深入本书这一部分的主要内容——持久性（persistence）——之前，我们先介绍**输入/输出（I/O）设备**（input/output device）的概念，并说明操作系统可能会怎样和这类设备打交道。当然，I/O 对计算机系统非常关键：想象一个没有任何输入的程序（每次运行结果都一样）；再想象一个没有任何输出的程序（那它运行的意义何在?）。显然，要让计算机系统有意思，输入和输出缺一不可。所以，本章要解决的问题是：

关键问题：如何把 I/O 集成到系统中
I/O 应该怎样集成到系统中？有哪些通用机制？怎样让它们高效?

## 36.1 系统架构

我们先来看一张典型系统的“经典”示意图（图 36.1，第 2 页）。图中有一个 CPU，它通过某种**内存总线**（memory bus）或互连线与系统的主存相连。一些设备通过一条通用 **I/O 总线**（I/O bus）连到系统上，在许多现代系统中，这条总线（bus）是 **PCI**（或它的众多衍生版本之一）；显卡和其他一些高性能 I/O 设备可能就接在这里。最后，再往下是一条或多条我们所说的**外围总线**（peripheral bus），比如 **SCSI**、**SATA** 或 **USB**。它们把慢速设备连到系统上，包括**磁盘**、**鼠标**和**键盘**。

你可能会问：为什么需要这样的分层结构? 简单地说：物理规律和成本。总线越快，就必须越短；因此，高性能的内存总线（memory bus）上没有多少空间插设备之类的东西。此外，把总线设计成高性能的，代价相当高。所以，系统设计者采用了这种分层的方法：要求高性能的部件（比如显卡）离 CPU 近一些，性能要求较低的部件离得远一些。把磁盘和其他慢速设备放在外围总线（peripheral bus）上，好处有很多，尤其是可以在上面接大量的设备。

当然，现代系统越来越多地使用专门的芯片组和更快的点对点互连来提高性能。图 36.2（第 3 页）大致画出了 Intel Z270 芯片组的结构 [H17]。在最上面，CPU 与内存系统连接得最紧密，但它与显卡（从而也与显示器）之间也有一条高性能的连接，以支持游戏（噢，太可怕了!）和其他图形密集型应用。

CPU 通过 Intel 专有的 **DMI**（**Direct Media Interface**，直接媒体接口）连到一个 I/O 芯片上，其余设备则通过多种不同的互连线连到这个芯片上。在右边，一块或多块硬盘（hard

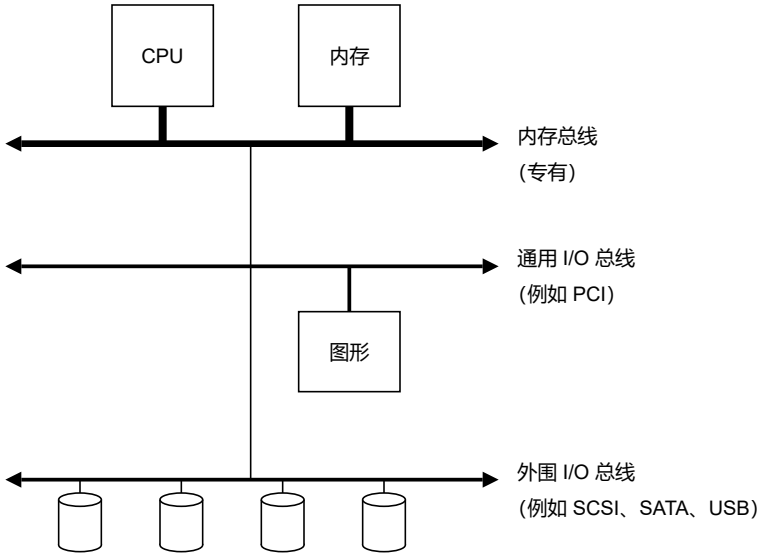



图 36.1：**典型的系统架构**

disk drive）通过 **eSATA** 接口连到系统上。**ATA**（**AT Attachment**，指为 IBM PC AT 提供连接）、后来的 **SATA**（**Serial ATA**，串行 ATA），以及现在的 eSATA（**external SATA**，外部 SATA），代表了几十年来存储接口的演进，每前进一步都提高了性能，以跟上现代存储设备（storage device）的步伐。

I/O 芯片下方是若干 **USB**（**Universal Serial Bus**，通用串行总线）连接，在这张图里，它们让键盘和鼠标可以接到计算机上。在许多现代系统中，USB 就用于这类低性能设备。

最后，在左边，其他更高性能的设备可以通过 **PCIe**（**Peripheral Component Interconnect Express**）连到系统上。在这张图里，网络接口就接在这里；更高性能的存储设备，比如 **NVMe** 持久存储设备（persistent storage device），也常常接在这里。

## 36.2 典型设备

现在我们来看一个典型的设备（不是真实的设备），借助它来理解：要让设备交互变得高效，需要哪些机制。从图 36.3（第 3 页）可以看到，一个设备有两个重要的组成部分。第一部分是它向系统其余部分呈现的硬件**接口**（interface）。和软件一样，硬件也必须提供某种接口，让系统软件能控制它的操作。因此，所有设备都有规定好的接口，以及典型交互所遵循的协议（protocol）。

任何设备的第二部分是它的**内部结构**（internal structure）。这一部分因设备而异，负责实现设备向系统呈现的抽象（abstraction）。非常简单的设备只用一块或几块硬件芯片来实现自身功能；更复杂的设备则会包含一个简单的 CPU、一些通用内存，以及其他设备专用的芯片来完成工作。例如，现代 RAID（廉价磁盘冗余阵列）控制器的功能，可能要靠几十万行**固件**（firmware，即硬件设备内部的软件）来实现。

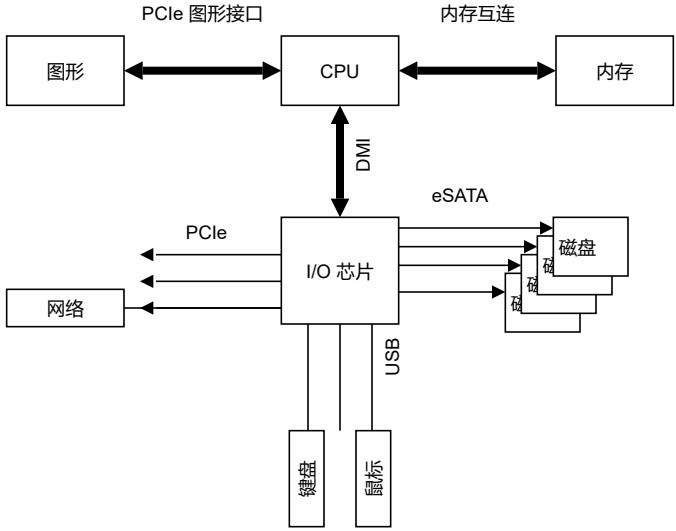



图 36.2：**现代系统架构**

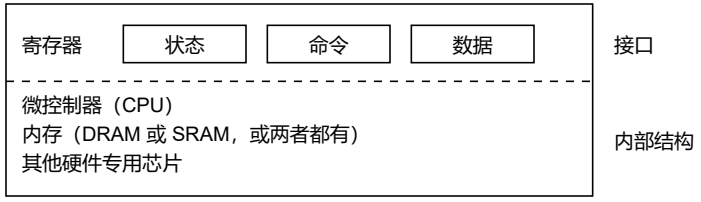



图 36.3：**典型设备**

## 36.3 典型协议

在上图中，（简化的）设备接口由三个寄存器组成：一个**状态**（status）寄存器，读取它可以看到设备的当前状态；一个**命令**（command）寄存器，用来告诉设备执行某项任务；还有一个**数据**（data）寄存器，用来向设备传送数据，或从设备取回数据。读写这些寄存器，操作系统就能控制设备的行为。

现在来看一次典型的交互：操作系统怎样让设备替它做事。协议如下：

```
While (STATUS == BUSY)
    ; // 等待，直到设备不忙
Write data to DATA register
Write command to COMMAND register
    （启动设备并执行命令）
While (STATUS == BUSY)
    ; // 等待，直到设备处理完你的请求
```

这个协议分四步。第一步，操作系统反复读取状态寄存器（status register），等待设备准备好接收命令；我们把这称为**轮询**（polling）设备（说白了就是问它在干什么）。第二步，操作系统把一些数据送到数据寄存器（data register）；可以想象，如果这是一块磁盘，要把一个磁盘块（disk block，比如 4 KB）传给设备，就需要执行多次写操作。主 CPU 参与数据搬运时（就像这个示例协议这样），就称为**程序控制 I/O**（programmed I/O，PIO）。第三步，操作系统把一条命令写入命令寄存器（command register）；这就隐含地告诉设备：数据已经就位，可以开始执行命令了。最后，操作系统再次循环轮询设备，等它完成（随后可能拿到一个表示成功与否的错误码）。

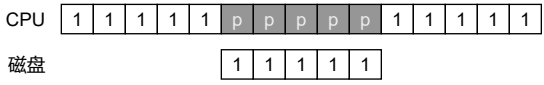
这个基本协议的优点是简单，而且能用。不过，它也有一些低效和不便之处。你可能会注意到的第一个问题是，轮询似乎很低效；具体来说，它把大量 CPU 时间白白花在等待（可能很慢的）设备完成工作上，而本来可以切换到另一个就绪（ready）的进程（process），把 CPU 用得更充分。

关键问题：如何避免轮询的开销
操作系统怎样才能不频繁轮询就检查设备状态，从而降低管理设备所需的 CPU 开销?

## 36.4 用中断降低 CPU 开销

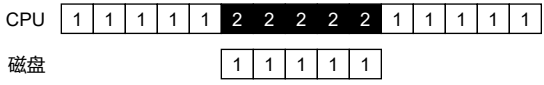
许多工程师多年前想出的改进办法，我们已经见过了：**中断**（interrupt）。操作系统不必反复轮询设备，而是可以发出请求，让调用进程进入睡眠（sleep），然后上下文切换（context switch）到另一个任务。设备最终完成操作时，会触发一个硬件中断，使 CPU 跳转到操作系统中一个预先确定的**中断服务程序**（interrupt service routine，ISR），或者更简单地说，**中断处理程序**（interrupt handler）。处理程序就是一段操作系统代码，它会完成这个请求（例如从设备读取数据，也许还有错误码），并唤醒（wake up）等待这次 I/O 的进程，让它按需继续执行。

因此，中断让计算与 I/O 可以**重叠**（overlap），这是提高利用率（utilization）的关键。下面这条时间线说明了问题所在：




在图中，进程 1 在 CPU 上运行了一段时间（CPU 那一行上重复的 `1` 表示这一点），然后向磁盘发出一个 I/O 请求，读取一些数据。如果没有中断，系统就只能空转，反复轮询设备的状态，直到 I/O 完成（用 `p` 表示）。磁盘处理完请求后，进程 1 终于可以再次运行。

如果改用中断，允许重叠，操作系统就可以在等待磁盘时做别的事：



提示：中断并不总是比轮询好

虽然中断让计算和 I/O 可以重叠，但它只对慢速设备才真正有意义。否则，处理中断和上下文切换的开销可能超过中断带来的好处。还有一些情况下，大量涌来的中断可能让系统过载，导致活锁（livelock）[MR96]；这时，轮询能让操作系统更好地控制调度，反倒派上了用场。

在这个例子中，磁盘处理进程 1 的请求时，操作系统在 CPU 上运行进程 2。磁盘请求完成时，会发生一个中断，操作系统唤醒进程 1，让它再次运行。这样，在中间这段时间里，CPU 和磁盘都得到了充分利用。

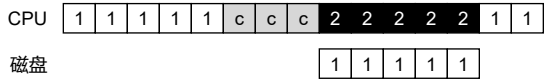
注意，使用中断并不总是最好的办法。例如，设想一个设备完成任务非常快：第一次轮询时，通常就会发现设备已经完成了任务。这时使用中断反而会拖慢系统：切换到另一个进程、处理中断、再切换回发出请求的进程，代价都很高。因此，如果设备很快，最好用轮询；如果设备很慢，最好用能实现重叠的中断。如果不知道设备的速度，或者它时快时慢，最好采用一种**混合**（hybrid）方式：先轮询一小会儿，如果设备还没完成，再改用中断。这种**两阶段**（two-phased）方法也许能兼得两者之长。

不使用中断的另一个原因出现在网络中 [MR96]。如果大量传入的数据包（packet）每个都产生一个中断，操作系统就可能陷入**活锁**（livelock），也就是发现自己只顾着处理中断，从不让用户级进程运行、真正去处理请求。例如，设想一台 Web 服务器因为登上了 Hacker News 榜首 [H18] 而遭遇负载激增。在这种情况下，最好偶尔用一下轮询，更好地控制系统中发生的事情，让 Web 服务器先处理一些请求，再回到设备那里查看有没有更多数据包到达。

另一种基于中断的优化是**合并**（coalescing）。采用这种做法时，需要触发中断的设备会先等一小会儿，再把中断发给 CPU。等待期间，其他请求可能很快也完成了，于是多个中断就可以合并成一次中断，从而降低中断处理的开销。当然，等得太久会增加请求的延迟（latency），这是系统中常见的权衡（trade-off）。Ahmad 等人的论文 [A+11] 对此做了很好的总结。

## 36.5 用 DMA 更高效地搬运数据

遗憾的是，我们的典型协议还有一个方面需要注意。具体来说，用程序控制 I/O（PIO）向设备传送一大块数据时，CPU 又一次被一件相当琐碎的任务压得喘不过气，浪费了大量时间和精力，而这些本来可以更好地用来运行其他进程。下面这条时间线说明了这个问题：



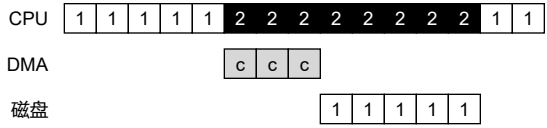

在这条时间线中，进程 1 正在运行，然后它想把一些数据写到磁盘上。于是它发起 I/O，而这次 I/O 必须显式地把数据从内存复制到设备，一次一个字（图中标为 c）。复制完成后，磁盘上的 I/O 才开始，CPU 终于可以拿去干别的事了。

 © 2008–25, ARPACI-DUSSEAU

关键问题：如何降低 PIO 的开销
使用 PIO 时，CPU 要花太多时间亲手在设备和内存之间搬运数据。怎样才能把这项工作卸下来，让 CPU 得到更有效的利用?

这个问题的解决办法就是我们所说的**直接内存访问**（Direct Memory Access，DMA）。DMA 引擎本质上是系统中一个非常专门的设备，它可以在设备和主存之间安排数据传输，而无需 CPU 过多干预。

DMA 的工作方式如下。例如，要把数据传给设备，操作系统会给 DMA 引擎编程，告诉它数据在内存中的位置、要复制多少数据，以及要发给哪个设备。到这一步，操作系统这边就没它的事了，可以去做别的工作。DMA 完成后，DMA 控制器会触发一个中断，操作系统由此得知传输已经完成。修改后的时间线如下：




从时间线可以看到，数据的复制现在由 DMA 控制器负责。由于这段时间 CPU 是空闲的，操作系统可以去做别的事，这里选择了运行进程 2。于是，在进程 1 再次运行之前，进程 2 能用上更多的 CPU 时间。

## 36.6 设备交互的方法

对执行 I/O 的效率问题有了一些认识之后，要把设备纳入现代系统，还得处理另外几个问题。你可能已经注意到了：我们还根本没说操作系统实际上是怎样与设备通信的！于是，问题来了：

关键问题：如何与设备通信
硬件应该怎样与设备通信？应该有专门的指令吗？还是有别的办法?

随着时间推移，发展出了两种主要的设备通信方法。第一种也是最古老的方法（IBM 大型机用了很多年），是使用显式的 **I/O 指令**（I/O instructions）。这些指令规定了操作系统向特定设备寄存器发送数据的方式，从而可以构建出上面描述的协议。

例如，在 x86 上，可以用 `in` 和 `out` 指令与设备通信。要向设备发送数据，调用者需要指定一个存放数据的寄存器，以及一个用来指明设备的特定端口（port）。执行这条指令，就会产生期望的行为。

这类指令通常是**特权**（privileged）指令。操作系统控制着设备，因此只有操作系统才能直接与设备通信。想象一下，如果任何程序都能读写磁盘，那就会一片混乱（一如既往），因为任何用户程序都可以利用这样的漏洞完全控制整台机器。

第二种与设备交互的方法叫作**内存映射 I/O**（memory-mapped I/O）。采用这种方法时，硬件让设备寄存器看起来就像内存位置一样。要访问某个寄存器，操作系统对相应地址执行一次加载（读）或存储（写）；硬件随后把这次加载或存储转给设备，而不是主存。

两种方法谈不上谁比谁有多大优势。内存映射（memory-mapped）方法的好处是不需要新增指令，但这两种方法如今都还在使用。

## 36.7 融入操作系统：设备驱动程序

我们要讨论的最后一个问题是：每种设备的接口都非常特殊，而我们又希望操作系统尽可能通用，那么怎样把设备融入操作系统呢？以文件系统（file system）为例。我们希望文件系统能构建在 SCSI 磁盘、IDE 磁盘、USB 闪存盘等设备之上，并且希望文件系统基本不用关心向这些不同类型的驱动器发出读写请求的种种细节。于是，我们的问题是：

关键问题：如何构建与设备无关的操作系统
怎样让操作系统的大部分保持与设备无关，从而对操作系统的主要子系统隐藏设备交互的细节？

这个问题要靠一项古老的技术来解决：**抽象**（abstraction）。在最底层，操作系统中必须有一部分软件详细了解设备是怎样工作的。我们把这部分软件称为**设备驱动程序**（device driver），所有与设备交互的细节都封装在其中。

以 Linux 的文件系统软件栈（file system software stack）为例，看看这种抽象怎样帮助操作系统的设计和实现。图 36.4 大致画出了 Linux 软件的组织方式。从图中可以看到，文件系统——当然还有它上面的应用程序——完全不知道自己用的是哪一类磁盘；它只是向通用块层（generic block layer）发出块（block）读写请求，通用块层再把请求转给合适的设备驱动程序，由驱动程序（driver）处理发出具体请求的细节。这张图虽然做了简化，但说明了这些细节是怎样对操作系统的大部分隐藏起来的。

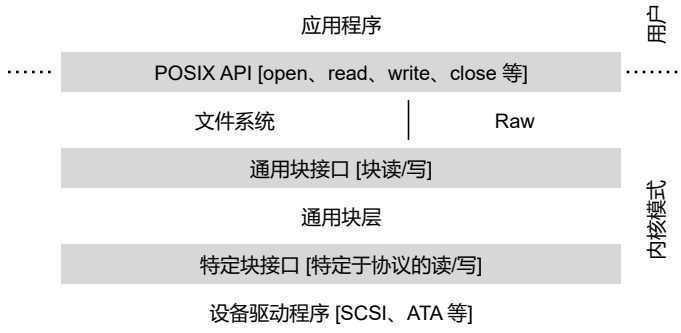



图 36.4：**文件系统栈**

图中还画出了直通设备的 **raw 接口**（raw interface）。有了它，一些特殊的应用程序可以不经过文件抽象（file abstraction），直接读写块，比如后面要介绍的**文件系统检查程序**（file-system checker）[AD14]，以及**磁盘碎片整理**（disk defragmentation）工具。大多数系统都提供这类接口，以支持这些底层的存储管理应用程序。

注意，上面看到的封装也可能有不好的一面。例如，如果某个设备有许多特殊功能，却只能向内核的其余部分呈现一个通用接口，那么这些特殊功能就用不上了。Linux 中的 SCSI 设备就遇到了这种情况：它们的错误报告非常丰富；但因为其他块设备（block device，例如 ATA/IDE）的错误处理要简单得多，所以上层软件收到的永远只是一个通用的 `EIO`（通用 I/O 错误）错误码，SCSI 可能提供的任何额外细节，文件系统都拿不到 [G08]。

有意思的是，由于你可能往系统里插的任何设备都需要设备驱动程序，久而久之，驱动程序在内核代码中占了极大的比例。对 Linux 内核的研究表明，超过 70% 的操作系统代码都在设备驱动程序中 [C01]；基于 Windows 的系统，这一比例很可能也相当高。所以，当有人告诉你操作系统有几百万行代码时，他们真正想说的是，操作系统有几百万行设备驱动程序代码。当然，在任何一台具体的机器上，这些代码大部分可能都不会运行（也就是说，同一时间只有少数几个设备连在系统上）。也许更让人沮丧的是，驱动程序常常由“业余人士”（而不是全职的内核开发者）编写，因此 bug 往往多得多，是导致内核崩溃的主要原因 [S03]。

## 36.8 案例研究：一个简单的 IDE 磁盘驱动程序

为了再深入一点，我们来快速看看一个真实的设备：IDE 磁盘驱动器（disk drive）[L94]。我们按参考资料 [W10] 的描述总结它的协议；还会瞄一眼 xv6 的源代码，看看一个能工作的 IDE 驱动程序的简单例子 [CK+08]。

IDE 磁盘向系统呈现的接口很简单，由四类寄存器组成：控制、命令块（command block）、状态和错误。（在 x86 上）用 `in` 和 `out` 这两条 I/O 指令（I/O instruction）读写特定的“I/O 地址”（比如下文的 `0x3F6`），就能访问这些寄存器。

假设设备已经初始化，与它交互的基本协议如下：

- **等待驱动器就绪**。读取状态寄存器（status register，0x1F7），直到驱动器处于 READY 且不处于 BUSY 状态。
- **把参数写入命令寄存器**。把扇区数（sector count）、要访问的扇区的逻辑块地址（LBA），以及驱动器号（主盘为 0x00，从盘为 0x10，因为 IDE 只允许接两个驱动器）写入命令寄存器（command register，0x1F2–0x1F6）。
- **启动 I/O**。把 READ|WRITE 命令写入命令寄存器（command register，0x1F7）。
- **传送数据（写操作）：**等待，直到驱动器状态为 READY 且 DRQ（驱动器请求数据）；然后把数据写到数据端口（data port）。
- **处理中断**。最简单的做法是每传送一个扇区（sector）处理一次中断；更复杂的做法允许批处理（batching），在整个传输完成时只产生最后一次中断。
- **错误处理**。每次操作后都读取状态寄存器（status register）。如果 ERROR 位为 1，就读取错误寄存器查看详情。

这个协议的大部分都可以在 xv6 的 IDE 驱动程序中找到（图 36.6，第 10 页）。这个驱动程序初始化之后，靠四个主要函数工作。第一个是 `ide_rw()`：如果有其他请求在等

```
控制寄存器:
  地址 0x3F6 = 0x08 (0000 1RE0): R=复位, E=0 表示"启用中断"

命令块寄存器:
  地址 0x1F0 = 数据端口
  地址 0x1F1 = 错误
  地址 0x1F2 = 扇区数
  地址 0x1F3 = LBA 低字节
  地址 0x1F4 = LBA 中字节
  地址 0x1F5 = LBA 高字节
  地址 0x1F6 = 1B1D TOP4LBA: B=LBA, D=驱动器
  地址 0x1F7 = 命令/状态

状态寄存器 (地址 0x1F7):
    7     6     5     4    3    2     1     0
  BUSY READY FAULT SEEK  DRQ CORR IDDEX ERROR

错误寄存器 (地址 0x1F1): (ERROR==1 时检查)
    7     6     5     4    3    2     1     0
  BBK   UNC   MC   IDNF  MCR ABRT T0NF  AMNF

  BBK  = 坏块
  UNC  = 不可纠正的数据错误
  MC   = 介质已更换
  IDNF = 未找到 ID 标记
  MCR  = 请求更换介质
  ABRT = 命令已中止
  T0NF = 未找到 0 号磁道
  AMNF = 未找到地址标记
```

图 36.5：**IDE 接口**

待，它就把请求放入队列（queue），否则直接发给磁盘（通过 `ide_start_request()`）；无论哪种情况，这个例程都会等待请求完成，调用进程随之进入睡眠。第二个是 `ide_start_request()`，用来把请求（如果是写操作，可能还有数据）发给磁盘；它调用 x86 的 `in` 和 `out` 指令，分别读写设备寄存器。启动请求的例程会用到第三个函数 `ide_wait_ready()`，在发出请求之前确保驱动器已经就绪。最后，发生中断时会调用 `ide_intr()`；它从设备读取数据（如果请求是读而不是写），唤醒等待 I/O 完成的进程，并且（如果 I/O 队列里还有请求）通过 `ide_start_request()` 启动下一次 I/O。

## 36.9 历史注记

在结束之前，我们简要介绍一下其中一些基本思想的起源。如果想了解更多，请阅读 Smotherman 精彩的总结 [S08]。

中断是一个很古老的思想，最早的机器上就已经有了。例如，20 世纪 50 年代初的 UNIVAC 就有某种形式的中断向量（interrupt vectoring），但这项功能具体是哪一年出现

```
static int ide_wait_ready() {
  while (((int r = inb(0x1f7)) & IDE_BSY) || !(r & IDE_DRDY))
    ; // 循环，直到驱动器不忙
  // 出错时返回 -1，否则返回 0
}

static void ide_start_request(struct buf *b) {
  ide_wait_ready();
  outb(0x3f6, 0);                 // 产生中断
  outb(0x1f2, 1);                 // 多少个扇区?
  outb(0x1f3, b->sector & 0xff); // LBA 放这里……
  outb(0x1f4, (b->sector >> 8) & 0xff);  // ……还有这里
  outb(0x1f5, (b->sector >> 16) & 0xff); // ……还有这里!
  outb(0x1f6, 0xe0 | ((b->dev&1)<<4) | ((b->sector>>24)&0x0f));
  if(b->flags & B_DIRTY){
    outb(0x1f7, IDE_CMD_WRITE);     // 这是一次写操作
    outsl(0x1f0, b->data, 512/4);  // 数据也一起传过去!
  } else {
    outb(0x1f7, IDE_CMD_READ);      // 这是一次读操作（没有数据）
  }
}

void ide_rw(struct buf *b) {
  acquire(&ide_lock);
  for (struct buf **pp = &ide_queue; *pp; pp=&(*pp)->qnext)
    ;                               // 遍历队列
  *pp = b;                          // 把请求加到队尾
  if (ide_queue == b)               // 如果队列为空
    ide_start_request(b);           // 把请求发给磁盘
  while ((b->flags & (B_VALID|B_DIRTY)) != B_VALID)
    sleep(b, &ide_lock);            // 等待完成
  release(&ide_lock);
}

void ide_intr() {
  acquire(&ide_lock);
  struct buf *b = ide_queue;
  if (!(b->flags & B_DIRTY) && ide_wait_ready() >= 0)
    insl(0x1f0, b->data, 512/4);    // 如果是读：取回数据
  b->flags |= B_VALID;
  b->flags &= ~B_DIRTY;
  wakeup(b);                        // 唤醒等待的进程
  if ((ide_queue = b->qnext) != 0) // 启动下一个请求
    ide_start_request(ide_queue);  //  （如果有的话）
  release(&ide_lock);
}
```

图 36.6：**xv6 的 IDE 磁盘驱动程序（简化版）**

的，现在已经说不清了 [S08]。可悲的是，计算机的历史才刚起步，我们就已经开始弄不清它的源头了。

至于哪台机器最先引入了 DMA 的思想，也有一些争论。例如，Knuth 等人认为是 DYSEAC（一台“移动”机器，在当时意味着可以用拖车拉着走），而另一些人认为 IBM SAGE 可能才是第一台 [S08]。无论如何，到 50 年代中期，已经出现了 I/O 设备直接与内存通信、完成后再中断 CPU 的系统。

这段历史很难追溯，因为这些发明都和一些真实的、有时还很冷门的机器联系在一起。例如，有人认为林肯实验室的 TX-2 是第一台拥有向量中断（vectored interrupt）的机器 [S08]，但这一点很难说清楚。

由于这些思想相对来说显而易见——想到让 CPU 在慢速 I/O 进行时去做别的事，并不需要爱因斯坦式的飞跃——也许我们执着于“谁是第一个”本身就是方向错了。可以肯定的是：人们在建造这些早期机器时，很明显就意识到需要 I/O 支持。中断、DMA 及相关思想，都是 CPU 快、设备慢这一本质特点的直接产物；如果你当时也在场，或许也会想到类似的点子。

## 36.10 小结

现在你应该对操作系统怎样与设备交互有了最基本的了解。我们介绍了两种提高设备效率的技术：中断和 DMA；还描述了访问设备寄存器的两种方法：显式 I/O 指令（I/O instruction）和内存映射 I/O（memory-mapped I/O）。最后，我们介绍了设备驱动程序的概念，说明了操作系统本身怎样封装底层细节，从而让操作系统的其余部分更容易做到与设备无关。

 © 2008–25, ARPACI-DUSSEAU

## 参考文献

[A+11] "vIC: Interrupt Coalescing for Virtual Machine Storage Device IO" by Irfan Ahmad, Ajay Gulati, Ali Mashtizadeh. USENIX '11. *A terrific survey of interrupt coalescing in traditional and virtualized environments.*

[AD14] "Operating Systems: Three Easy Pieces" (Chapters: Crash Consistency: FSCK and Journaling and Log-Structured File Systems) by Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau. Arpaci-Dusseau Books, 2014. *A description of a file-system checker and how it works, which requires low-level access to disk devices not normally provided by the file system directly.*

[C01] "An Empirical Study of Operating System Errors" by Andy Chou, Junfeng Yang, Benjamin Chelf, Seth Hallem, Dawson Engler. SOSP '01. *One of the first papers to systematically explore how many bugs are in modern operating systems. Among other neat findings, the authors show that device drivers have something like seven times more bugs than mainline kernel code.*

[CK+08] "The xv6 Operating System" by Russ Cox, Frans Kaashoek, Robert Morris, Nickolai Zeldovich. From: http://pdos.csail.mit.edu/6.828/2008/index.html. *See* `ide.c` *for the IDE device driver, with a few more details therein.*

[D07] "What Every Programmer Should Know About Memory" by Ulrich Drepper. November, 2007. Available: http://www.akkadia.org/drepper/cpumemory.pdf. *A fantastic read about modern memory systems, starting at DRAM and going all the way up to virtualization and cache-optimized algorithms.*

[G08] "EIO: Error-handling is Occasionally Correct" by Haryadi Gunawi, Cindy Rubio-Gonzalez, Andrea Arpaci-Dusseau, Remzi Arpaci-Dusseau, Ben Liblit. FAST '08, San Jose, CA, February 2008. *Our own work on building a tool to find code in Linux file systems that does not handle error return properly. We found hundreds and hundreds of bugs, many of which have now been fixed.*

[H17] "Intel Core i7-7700K review: Kaby Lake Debuts for Desktop" by Joel Hruska. January 3, 2017. www.extremetech.com/extreme/241950-intels-core-i7-7700k-reviewed-kaby-lake-debuts-desktop. *An in-depth review of a recent Intel chipset, including CPUs and the I/O subsystem.*

[H18] "Hacker News" by Many contributors. Available: https://news.ycombinator.com. *One of the better aggregators for tech-related stuff. Once back in 2014, this book became a highly-ranked entry, leading to 1 million chapter downloads in just one day! Sadly, we have yet to re-experience such a high.*

[L94] "AT Attachment Interface for Disk Drives" by Lawrence J. Lamers. Reference number: ANSI X3.221, 1994. Available: ftp://ftp.t10.org/t13/project/d0791r4c-ATA-1.pdf. *A rather dry document about device interfaces. Read it at your own peril.*

[MR96] "Eliminating Receive Livelock in an Interrupt-driven Kernel" by Jeffrey Mogul, K. K. Ramakrishnan. USENIX '96, San Diego, CA, January 1996. *Mogul and colleagues did a great deal of pioneering work on web server network performance. This paper is but one example.*

[S08] "Interrupts" by Mark Smotherman. July '08. Available: http://people.cs.clemson.edu/~mark/interrupts.html. *A treasure trove of information on the history of interrupts, DMA, and related early ideas in computing.*

[S03] "Improving the Reliability of Commodity Operating Systems" by Michael M. Swift, Brian N. Bershad, Henry M. Levy. SOSP '03. *Swift's work revived interest in a more microkernel-like approach to operating systems; minimally, it finally gave some good reasons why address-space based protection could be useful in a modern OS.*

[W10] "Hard Disk Driver" by Washington State Course Homepage. Available online at this site: http://eecs.wsu.edu/~cs460/cs560/HDdriver.html. *A nice summary of a simple IDE disk drive's interface and how to build a device driver for it.*